

Enterprise Tiny Agent Framework Design Guide

Purpose

This document defines a practical architecture and implementation approach for building a collection of small, focused AI agents using Java and Spring Boot.

The goal is to create agents that:

- Own a single business capability
 - Are deterministic and observable
 - Use tools rather than hallucinated knowledge
 - Can be independently tested and deployed
 - Support local and hosted LLMs
 - Scale without becoming an unmaintainable multi-agent system
-

Design Principles

Principle 1: Capability-Centric Design

Agents should be designed around business capabilities, not technologies.

Bad

```
DuckDB Agent  
Calcite Agent  
Filesystem Agent
```

Good

```
Metadata Discovery Agent  
SQL Authoring Agent  
Job Operations Agent  
Research Agent  
Documentation Agent
```

The business capability should remain stable even if implementation technologies change.

Principle 2: Single Responsibility

Every agent must answer:

What is the one thing I do exceptionally well?

Examples:

Agent	Responsibility
Metadata Agent	Dataset discovery
SQL Agent	SQL generation and validation
Operations Agent	Job management
Research Agent	Information gathering and summarization
Documentation Agent	System explanation

If an agent description contains the word "and", consider splitting it.

Principle 3: Tools Perform Work

The model should decide.

Tools should execute.

```
Agent
  ↓
Select Tool
  ↓
Tool Executes
  ↓
Agent Responds
```

Never allow the model to directly manipulate systems.

Principle 4: Small Context

Agents should receive only relevant context.

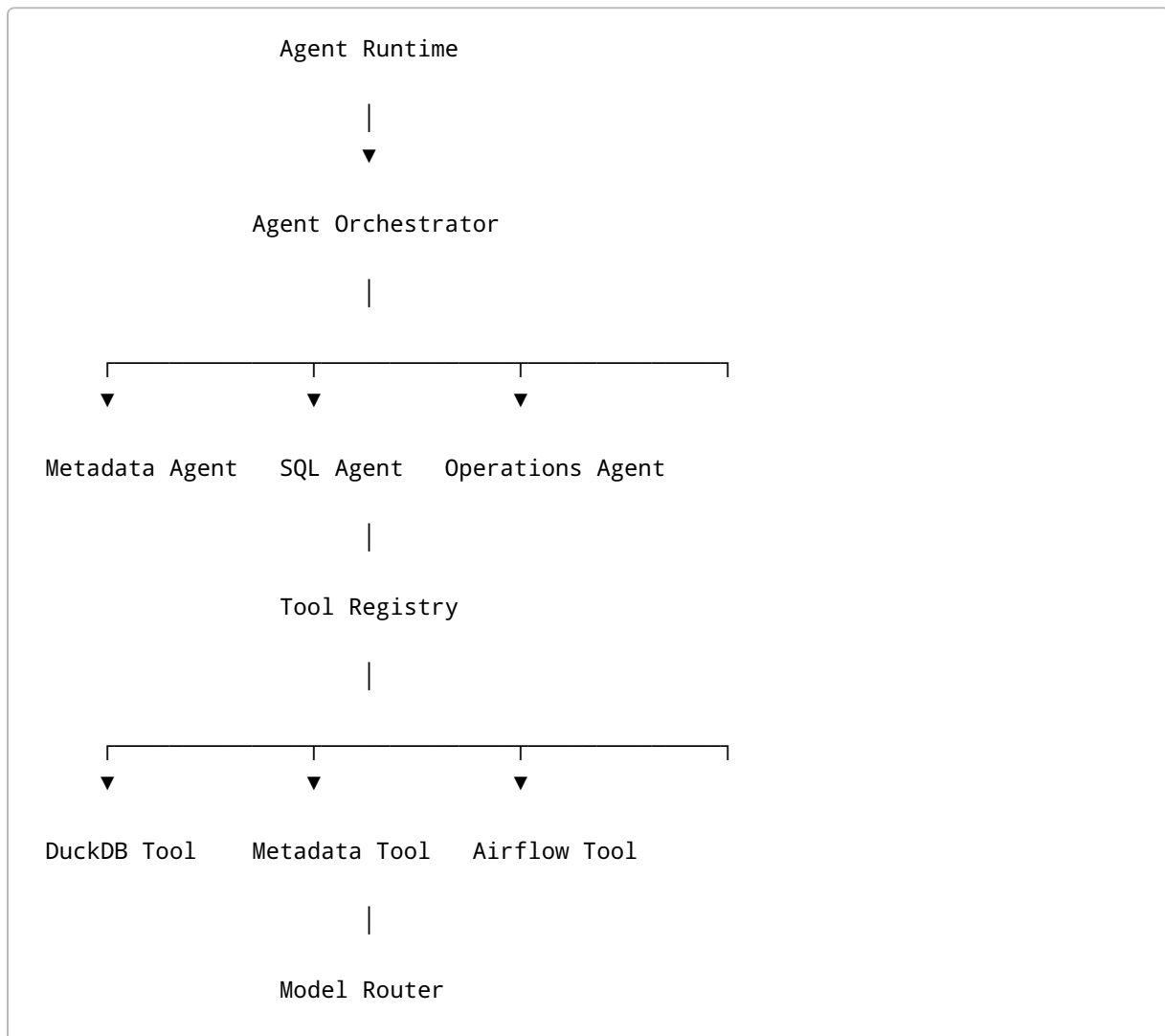
Avoid:

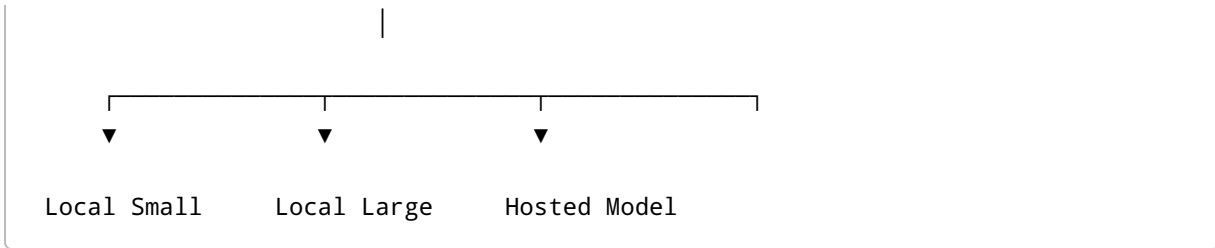
Entire conversation history
Entire metadata catalog
Entire documentation repository

Prefer:

Request
Retrieved Context
Tool Results
Session Memory

High Level Architecture





Agent Lifecycle

Every agent should follow the same lifecycle.

```
Request
↓
Initialize
↓
Validate
↓
Build Context
↓
Plan
↓
Execute
↓
Generate Response
↓
Audit
↓
Complete
```

Core Runtime Interfaces

Agent

```
public interface Agent {

    AgentResponse execute(
        AgentRequest request
    );
}
```

```
}
```

AbstractAgent

```
public abstract class AbstractAgent
    implements Agent {

    @Override
    public AgentResponse execute(
        AgentRequest request) {

        AgentExecutionContext context =
            initialize(request);

        validate(context);

        buildContext(context);

        AgentPlan plan =
            plan(context);

        AgentExecutionResult result =
            executePlan(context, plan);

        AgentResponse response =
            generateResponse(
                context,
                result);

        audit(context, result);

        return response;
    }

    protected abstract void validate(
        AgentExecutionContext context);

    protected abstract void buildContext(
        AgentExecutionContext context);

    protected abstract AgentPlan plan(
        AgentExecutionContext context);
```

```
protected abstract AgentExecutionResult executePlan(
    AgentExecutionContext context,
    AgentPlan plan);

protected abstract AgentResponse generateResponse(
    AgentExecutionContext context,
    AgentExecutionResult result);
}
```

Execution Context

The context is the primary state object.

```
public class AgentExecutionContext {

    private String requestId;

    private String agentId;

    private String userInput;

    private Map<String, Object> attributes;

    private List<ToolResult> toolResults;

    private AgentMemory memory;

}
```

Rules:

- No global variables
- No hidden state
- Everything travels through context

Planning Layer

Agents should create plans before execution.

```
public class AgentPlan {  
  
    private List<PlannedAction> actions;  
  
}
```

Example:

```
SEARCH_METADATA  
VALIDATE_SCHEMA  
GENERATE_SQL
```

Benefits:

- Easier debugging
- Better observability
- Deterministic execution

Tool Framework

Tool Interface

```
public interface Tool {  
  
    String name();  
  
    ToolResult execute(  
        ToolRequest request);  
  
}
```

Tool Registry

```
public interface ToolRegistry {  
  
    Tool getTool(  
        String name);  
  
}
```

```
}
```

Example Tools

```
SearchMetadataTool  
  
GetDatasetTool  
  
GetLineageTool  
  
ExecuteSqlTool  
  
ValidateSqlTool  
  
GetJobStatusTool  
  
RestartJobTool  
  
WebSearchTool
```

Model Layer

Model Provider

```
public interface ModelProvider {  
  
    LlmResponse generate(  
        LlmRequest request);  
  
}
```

Implementations:

```
OllamaProvider  
  
OpenAIProvider  
  
LMStudioProvider
```


vLLMProvider

Model Router

```
public interface ModelRouter {  
  
    ModelProvider route(  
        AgentExecutionContext context);  
  
}
```

Example routing:

```
SQL Agent  
  → Local Qwen 4B  
  
Research Agent  
  → Local Qwen 14B  
  
Critical Requests  
  → Hosted Model
```

Memory Layer

Most agents should have minimal memory.

```
public interface MemoryProvider {  
  
    AgentMemory load(  
        String sessionId);  
  
    void save(  
        AgentMemory memory);  
  
}
```

Implementations:

InMemory

Redis

PostgreSQL

Agent Interceptors

Used for auditing and monitoring.

Before Execution

```
public interface AgentInterceptor {  
  
    void beforeExecution(  
        AgentExecutionContext context);  
}
```

After Execution

```
public interface AgentInterceptor {  
  
    void afterExecution(  
        AgentExecutionContext context,  
        AgentResponse response);  
}
```

Use for:

- Metrics
- Security
- Auditing
- Logging

Observability

Every execution should emit events.

```
AGENT_STARTED  
  
CONTEXT_BUILT  
  
PLAN_CREATED  
  
TOOL_SELECTED  
  
TOOL_EXECUTED  
  
MODEL_CALLED  
  
AGENT_COMPLETED  
  
AGENT_FAILED
```

Store:

```
Request ID  
Agent ID  
Model Used  
Tools Invoked  
Execution Time  
Token Usage  
Result
```

Agent Definition

All agents should be declaratively described.

```
id: metadata-agent  
  
description: Dataset discovery agent  
  
preferredModel: qwen-4b  
  
fallbackModel: qwen-14b  
  
tools:  
  - metadata-search  
  - metadata-lineage
```

```
confidenceThreshold: 0.80
```

```
maxExecutionTime: 30s
```

Agent Template

Every new agent should define:

```
name:
```

```
businessCapability:
```

```
successMetric:
```

```
allowedTools:
```

```
forbiddenTools:
```

```
inputSchema:
```

```
outputSchema:
```

```
preferredModel:
```

```
fallbackModel:
```

```
confidenceThreshold:
```

```
maxExecutionTime:
```

If this definition exceeds one page, the agent is probably too large.

Example Metadata Agent

Capability:

```
Dataset Discovery
```

Tools:

Metadata Search Tool
Dataset Detail Tool
Lineage Tool

Output:

```
{
  "datasets": [
    {
      "name": "subscriber_usage",
      "confidence": 0.95
    }
  ]
}
```

Orchestrator

Single entry point.

```
public interface AgentOrchestrator {

    AgentResponse process(
        AgentRequest request);

}
```

Responsibilities:

- Intent classification
- Agent selection
- Execution tracking
- Result aggregation

Agents should not directly call other agents.

Recommended Initial Agents

Metadata Agent

Responsibilities:

- Dataset discovery
 - Schema lookup
 - Lineage lookup
-

SQL Agent

Responsibilities:

- SQL generation
 - SQL validation
 - Query explanation
-

Operations Agent

Responsibilities:

- Job status
 - Job restart
 - Execution monitoring
-

Documentation Agent

Responsibilities:

- Explain system behavior
 - Explain metadata
 - Explain errors
-

Research Agent

Responsibilities:

- Search web

- Read sources
 - Summarize findings
-

Recommended Package Structure

```
agent-runtime
├── agent-api
├── agent-core
├── tool-api
├── tool-runtime
├── llm-api
├── model-router
├── memory-api
├── audit-api
├── orchestrator
└── spring-boot-starter-agent
```

Future Enhancements

After the platform stabilizes:

1. Retrieval-Augmented Generation (RAG)
2. Agent capability registry
3. MCP adapters
4. Workflow execution engine
5. Agent marketplace
6. Human approval workflows

Avoid introducing these until the core runtime is mature.

Final Rule

An agent should be:

- Small
- Observable
- Deterministic
- Tool-driven
- Capability-focused

When in doubt:

Split the agent. Split the tool. Reduce the context. Simplify the responsibility.

The simplest agent that reliably solves a business problem is usually the correct design.